

Parciales MD

Parcial 4-11-24

1) Las unidades postales (UP) son sucursales de correo que se ocupan habitualmente de recibir y entregar paquetes. En particular, la UP 16 sólo se ocupa de entregar paquetes a las personas que se presentan a retirarlos. Los clientes son atendidos por orden de llegada por el primer empleado que se encuentre libre. Al solicitar atención, el cliente le muestra un código QR al empleado que lo atiende, el cual le entrega el paquete como respuesta. Implemente un programa que permita modelar el funcionamiento de la UP 16 usando **PMA**, asumiendo que hay N clientes que retiran un único paquete y 3 empleados para atenderlos. Además, asuma que no habrá códigos QR erróneos ni paquetes faltantes. **Notas:** la función *obtenerQR(idCliente)* retorna el código QR asociado al identificador de cliente recibido como entrada. De forma similar, *obtenerPaquete(qr)* retorna el paquete asociado al QR recibido como entrada.

PMA

```
chan SolicitarAtencion(integer, QR);
chan esperarPaquete[N](Paquete);

process Cliente[id = 0 to N-1] {
    Paquete paquete;
    QR qr = obtenerQR(id);
    send SolicitarAtencion(id, qr);
    receive esperarPaquete[id](paquete);
}

process Empleado[id = 0 to 2] {
    Paquete paquete;
    QR qr;
    int idC;
    while true {
        receive SolicitarAtencion(idC, qr);
        paquete = obtenerPaquete(qr);
        send esperarPaquete[id](paquete);
    }
}
```

PMS

```

process Cliente[id = 0 to N-1] {
    Paquete paquete;
    QR qr = obtenerQR(id);
    Coordinador!SolicitarAtencion(id, qr);
    Empleado[*]?EsperarPaquete(paquete);
}

process Coordinador {
    idC, idE: integer;
    QR qr;
    Cola cola;
    do Cliente[*]?SolicitarAtencion(idC,qr) → {
        cola.push(idC, qr);
    }
    [] !empty(cola); Empleado[*]?EstoyLibre(idE) → {
        cola.pop(idC, qr);
        Empleado[idE]!RecibirPaquete(idC, qr);
    }
od
}

process Empleado[id = 0 to 2] {
    Paquete paquete;
    QR qr;
    Integer idC;
    while true {
        Coordinador!EstoyLibre(id);
        Coordinador?RecibirCliente(idC, qr);
        paquete = obtenerPaquete(qr);
        Cliente[idC]!EsperarPaquete(paquete);
    }
}

```

3) El problema *WordCount* consiste en calcular la cantidad de veces que aparece una palabra determinada en una porción, documento o archivo de texto. Implemente una solución a *WordCount* en ADA, considerando que se cuenta con una base de datos de texto distribuida en N partes. Además, existe un administrador que determina la palabra que se desea buscar y se la envía a las N tareas contadoras para que la busquen en la parte de la base de texto que poseen. Finalmente, el administrador calcula la cantidad total de apariciones. **Notas:** las tareas contadoras cuentan con la función *contarPalabras(pal)*, la cual retorna la cantidad de veces que aparece la palabra *pal* en su parte de la base de datos. El administrador cuenta con la función *obtenerPalabra()* que retorna la palabra a buscar. El problema se resuelve una única vez y su solución debe maximizar la concurrencia.

```
procedure WordCount is

task Administrador is
    entry recibirCantidad(cant: in int);
    entry buscarPalabra(palabra: out string);
end Administrador;

task type Contador is
end Contador;
vectorContadores: array[1..N] of Contador;

task body Administrador is
    string pal = obtenerPalabra();
    int total = 0;
begin;
    for i in 1..2*N loop
        SELECT
            accept buscarPalabra(palabra: out string) do
                palabra = pal;
            end buscarPalabra;
        OR
            accept recibirCantidad(cant: in int) do
                total+= cant;
            end recibirCantidad;
        end loop;
    end Administrador;

task body Contador is
    String palabra;
```

```
begin
  Administador.buscarPalabra(palabra);
  cant = contarPalabras(palabra);
  Administrador.recibirCantidad(cant);
end Contador;
```

```
begin
  null;
end WordCount;
```